

ASSIGNMENT 1

NAME: KATHIRSELVAN M

ROLL NO: 71772118L06

CLASS: B.TECH IT - 4th_Year

SUBJECT: DEV-OPS

GIT & GITHUB

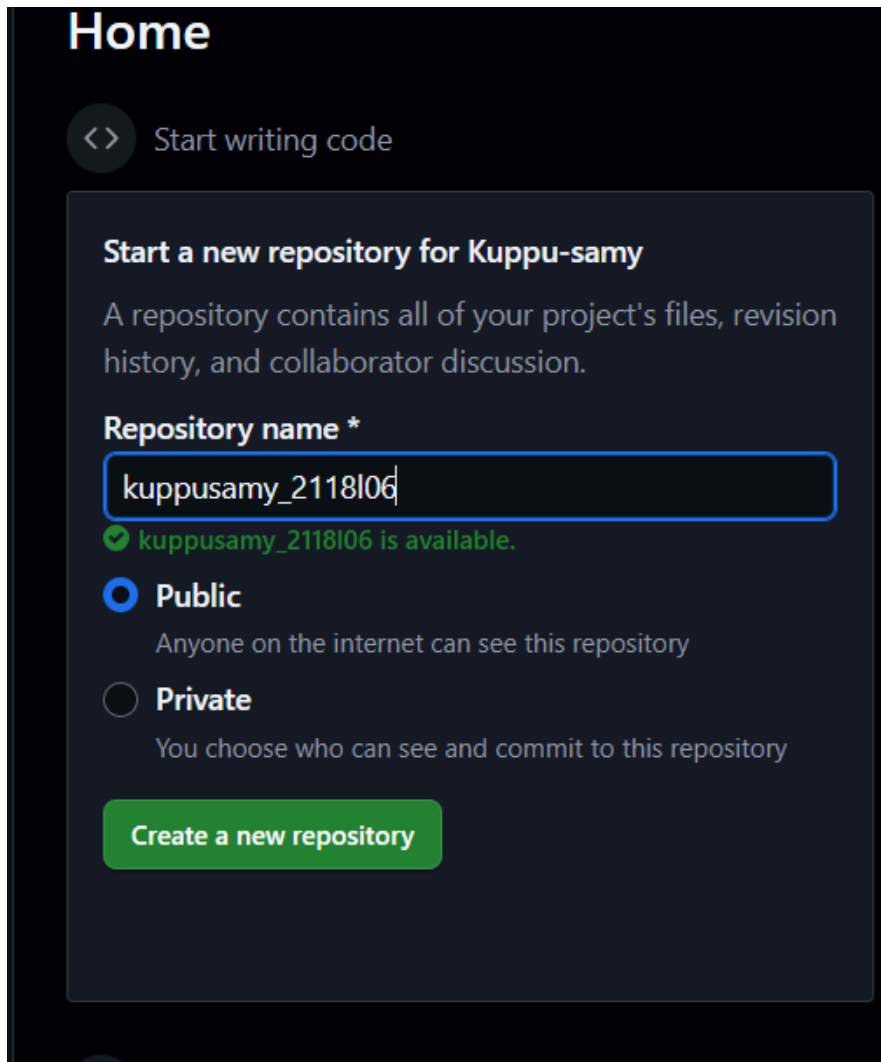
1. To download a remote repository to your local machine, use the following command: Example

git clone https://github.com/Kuppu-samy/kuppusamy_2118l06.git

```
psycho@LAPTOP-5U7P3J86 MINGW32 /d/gitdemo (master)
$ git clone https://github.com/Kuppu-samy/kuppusamy_2118l06.git
Cloning into 'kuppusamy_2118l06'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

The above screenshot shows a successful cloning of a repository containing 3 objects.

2. Created a repository named kathirselvan_2118l06 on GitHub.

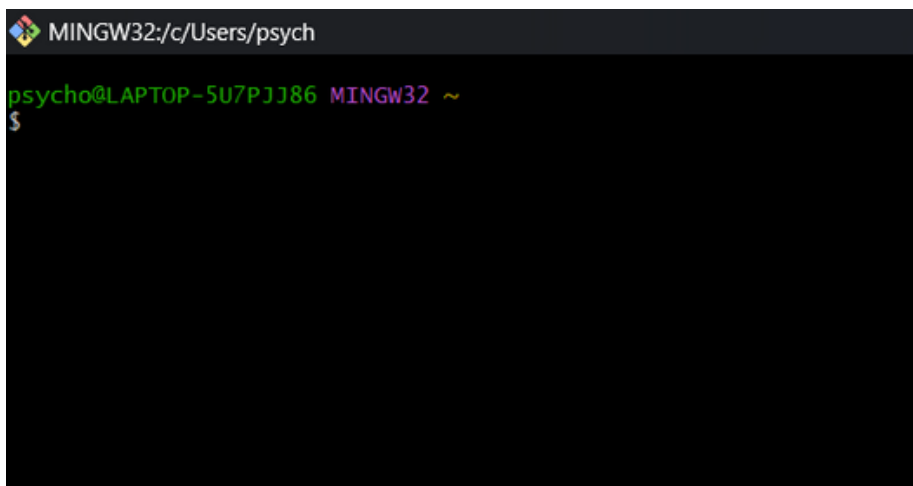
The screenshot shows the GitHub 'Home' page with a dark theme. At the top, there's a 'Home' header and a button with a code icon and the text 'Start writing code'. Below this is a card titled 'Start a new repository for Kuppu-samy'. The card contains a description: 'A repository contains all of your project's files, revision history, and collaborator discussion.' followed by a 'Repository name *' label and a text input field containing 'kuppusamy_2118l06'. Below the input field is a green checkmark and the text 'kuppusamy_2118l06 is available.' There are two radio button options: 'Public' (selected) with the description 'Anyone on the internet can see this repository' and 'Private' with the description 'You choose who can see and commit to this repository'. At the bottom of the card is a green button labeled 'Create a new repository'.

Created a file named index.html under ass folder using touch command.

Here are a few Git Bash commands:

- `cd <folder_name >`
- `mkdir <new folder>`
- `touch filename`
- `rm filename`
- `clear`

These basic commands will help you navigate, create files, and manage directories in Git Bash.



```
MINGW32:/c/Users/psych
psycho@LAPTOP-5U7PJ386 MINGW32 ~
$
```

After creating the file, check the folder and enter a text in that file.

Command : `git init`

To initialize a new Git repository in the current directory we use the “`git init`”



```
MINGW32:/c/Users/psych
psycho@LAPTOP-5U7PJ386 MINGW32 ~
$ git init
Initialized empty Git repository in C:/Users/psych/.git/
psycho@LAPTOP-5U7PJ386 MINGW32 ~ (master)
$
```

This sets up a `.git` directory in your project folder, making it a Git repository.

Command : `git add “filename” (or) git add .`

Next we used the “`git add`” command to stage the files to be included in the next commit. The “`.`” operator denotes all the files in the directory.

Command : `git commit -m "Your descriptive commit message"`

Then to save the staged changes to the repository with a commit message, we used “`git commit`” command. The `-m` parameter is used to show a message after committing the changes .

```
MINGW32/d/gitdemo
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo
$ git init
Initialized empty Git repository in D:/gitdemo/.git/
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ git add .
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ git config --global user.name "Kuppu-samy"
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ AC
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ A[[200~git config --global user.name "Kuppu-samy"
bash: $'\E[200~git': command not found
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ git config --global user.email "ksamy2429@gmail.com"
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ git commit -m "Initial commit"
[master (root-commit) 0f1e226] Initial commit
1 file changed, 12 insertions(+)
create mode 100644 index.html
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ git remote add origin https://github.com/Kuppu-samy/kuppusamy_2118106.git
psycho@LAPTOP-5U7PJ386 MINGW32 /d/gitdemo (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 318 bytes | 318.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Kuppu-samy/kuppusamy_2118106.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Command : git remote add origin https://github.com/user/repository.git

The above screenshot show all the step 1 to 5.

The “git remote add origin” command is used to link our local Git repository to a remote repository.

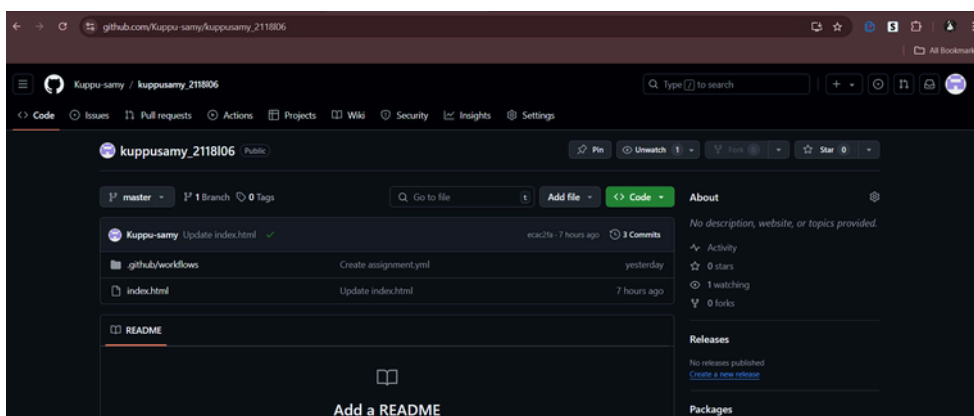
Command : git commit -m "Your descriptive commit message"

To check the **current state** of your working directory and staging area, we used the command “git status.”

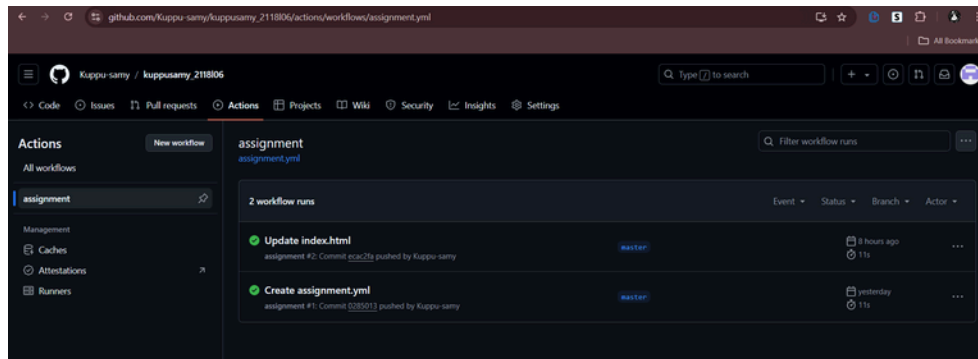
Then we used the “git push” command to upload our committed changes from the local repository to the remote repository. We used “git push -u origin master” command to set the upstream branch (which tells Git to track the remote branch).

Command : git push -u origin master

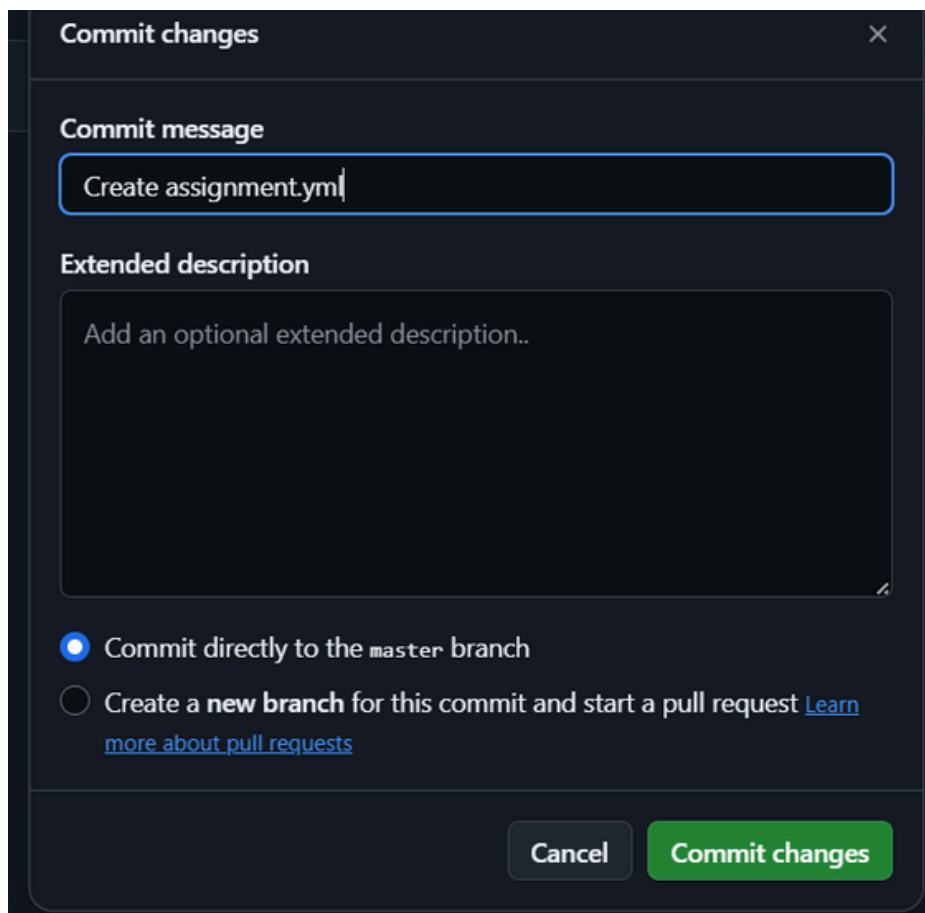
After pushing, we can check our remote repository for the file which we committed and pushed.



GITHUB ACTIONS: creating

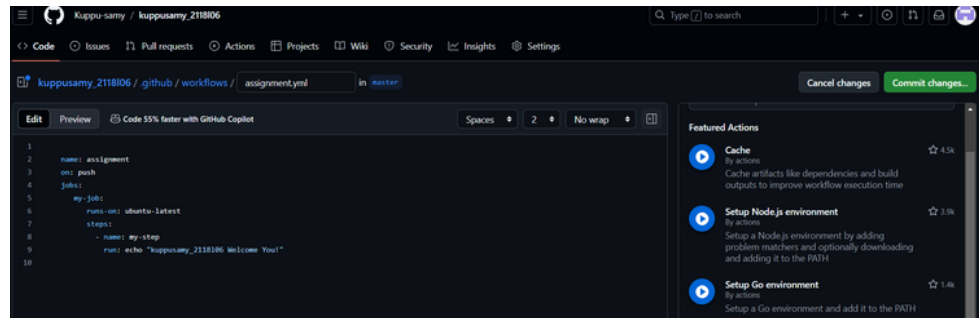


Created a `.github/workflows/` /
GIT Actions start with “.” like `.github` . Then created “yml” file .

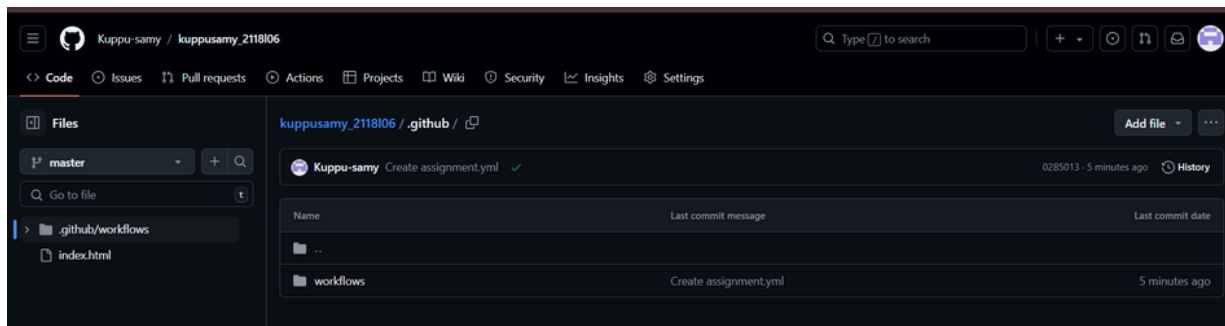


This is a basic YAML configuration with three key-value pairs: the app's name, version, and whether it's enabled. It's a simple format, but it could easily be expanded to add more complexity if needed.

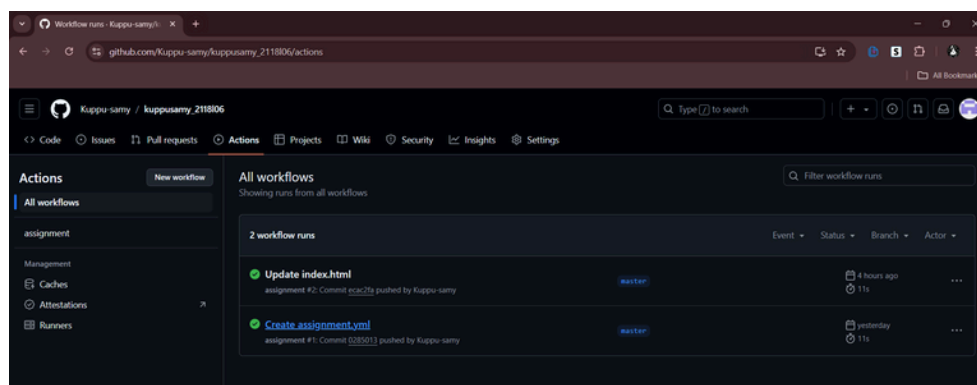
3. Created a **YAML workflow** file named “assignment.yml” to define when and how to run jobs.



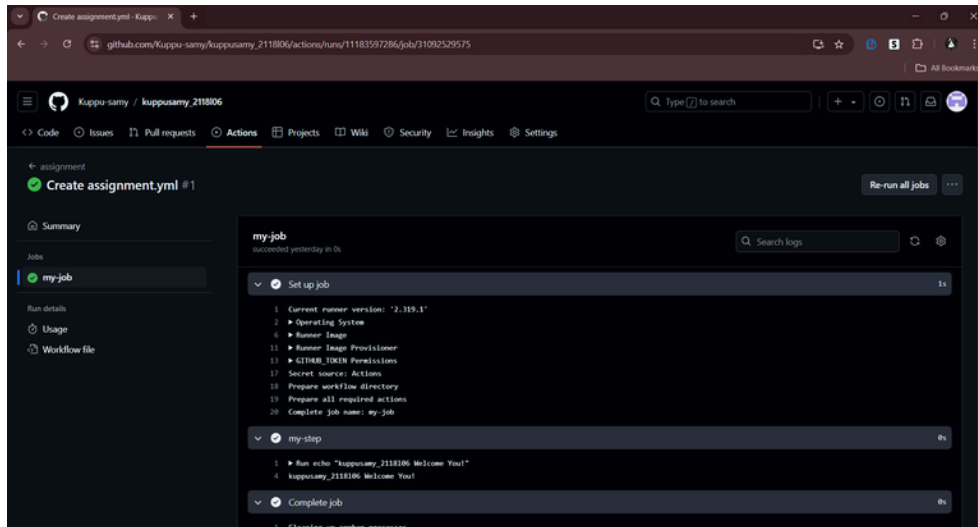
4. Committed and pushed the workflow file.



5. Now, we can monitor the workflow execution in the Actions tab on GitHub.



- Ensure that your GitHub repository has permissions set to allow GitHub Actions to push changes.
- You may need to set up a personal access token with write permissions if you encounter permission issues.

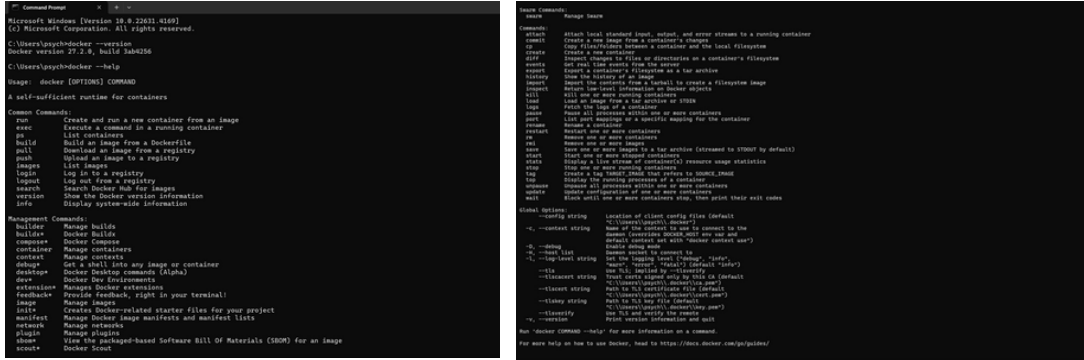


above screenshots shows the workflow execution.

GitHub Actions automates workflows directly in your repository, allowing you to run tasks like testing and deployment based on events such as pushes or pull requests. It enhances productivity by enabling customizable and reusable workflows that streamline the software development process.

DOCKER

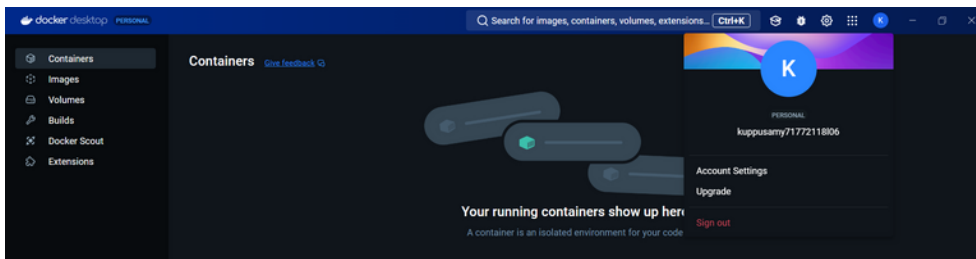
1. To check Docker installation we run the following command : **docker --help**



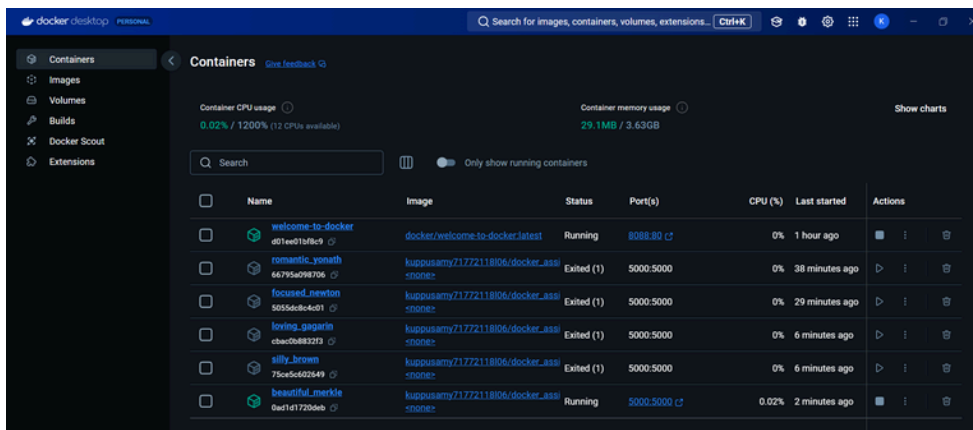
2. Created a repository named “appdocker” in which I created a HTML file and a

Dockerfile.File name is index.html

3.docker login -u **<your_dockerhub_username>**, after successfully logging in it will show a message like “Login succeeded”.



sample default image welcome -to-docker push in container



4. To **build a Docker image** I used the following command:

```
docker build -t kuppusamy71772118l06/docker_assignment .
```

This command creates a Docker image named kuppusamy71772118l06 with tag name :latest

```
D:\docker_assignment>docker build -t kuppusamy71772118l06/docker_assignment .
[+] Building 11.9s (0/10) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 555B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:49f94609e5a997dc16886a66ac9664591854031d48e375945a9dbfd1d53abbc
=> resolve docker.io/library/python:3.9-slim@sha256:49f94609e5a997dc16886a66ac9664591854031d48e375945a9dbfd1d53abbc
=> [internal] load build context
=> transferring context: 763B
=> CACHED [2/5] WORKDIR /usr/src/app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:57f8cceb7fd655b819afa6294ecd836972582564dc173eaf8af00c704c3119b6
=> exporting config sha256:14d268c3c1e9ce8426deafbb47440894a9ef6e0c9bb4fb05a31570c1a643e4c2
=> exporting attestation manifest sha256:f83b7181af02be5277d08171a3e11e38993a54ebdc238f308198212958bf73ba
=> exporting manifest list sha256:408f62a2878d93bae7c62e8ecdd7b8d4114f444898e5a40a344d2868dc8d9
=> naming to docker.io/kuppusamy71772118l06/docker_assignment:latest
=> unpacking to docker.io/kuppusamy71772118l06/docker_assignment:latest
```

5. To **docker run image** I used the following command :

```
docker run -p 5050:5000 kuppusamy71772118l06/docker_assignment
```

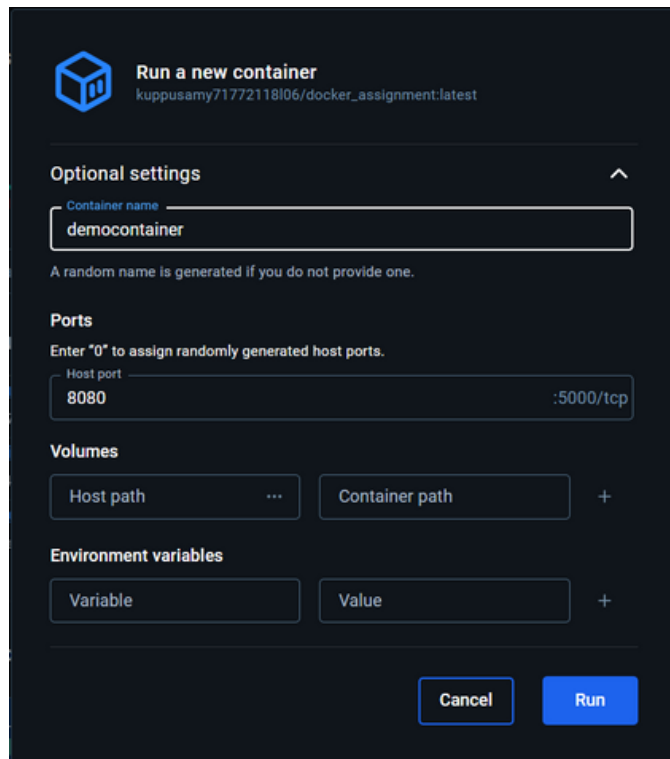
```
D:\docker_assignment>docker run -p 5000:5000 kuppusamy71772118l06/docker_assignment
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on all addresses (0.0.0.0)
  WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.3:5000 (Press CTRL+C to quit)
172.17.0.1 - - [05/Oct/2024 10:10:28] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [05/Oct/2024 10:10:28] "GET /favicon.ico HTTP/1.1" 404 -
```

6. docker run -p 5050:5000 kuppusamy71772118l06/docker_assignment
webpage sever any search engine use here put url localhost:5000



6. To test the container locally, I opened a web browser and entered <http://localhost:8080>.

run the container port and container name should provide
That container name: democontainer
port set : 8080



Run a new container
kppusamy71772118l06/docker_assignment:latest

Optional settings

Container name
democontainer

A random name is generated if you do not provide one.

Ports
Enter "0" to assign randomly generated host ports.
Host port: 8080 :5000/tcp

Volumes
Host path ... Container path +

Environment variables
Variable Value +

Cancel Run

7. Set the container name and port value here picture above. Then check while any browser Webpage **URL localhost:8080** here
before, I change the index.html file image statement here, see to display on the webpage output statement change. compare to localhost:5000



8.To check the list of images and containers, I used the following command: dockerimages and docker ps (for containers).

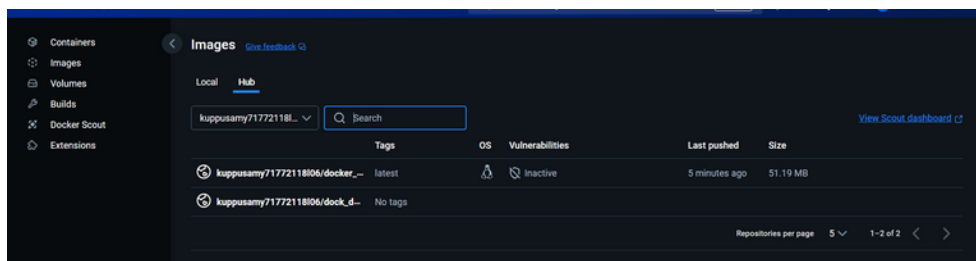
```
D:\docker_assignment>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
d81ee01bf8c9   docker/welcome-to-docker:latest    "/docker-entrypoint..." 2 hours ago    Up 2 hours    0.0.0.0:8080->80/tcp     welcome-to-docker

D:\docker_assignment>docker images
REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
kuppusamy71772118106/docker_assignment   latest      2758e5cd605c  12 hours ago  207MB
sivas04/siva_71772118502                 latest      f4bcc3e7b1f9  17 hours ago  207MB
docker/welcome-to-docker                 latest      eedaff45e3c7  11 months ago 29.5MB

D:\docker_assignment>
```

9.Login to Docker Hub using the following command to push the image to Docker Hub:

```
D:\docker_assignment>docker push kuppusamy71772118106/docker_assignment:latest
The push refers to repository [docker.io/kuppusamy71772118106/docker_assignment]
4c0965d39195: Pushed
1f899d4781dc: Pushed
fdeec83abba: Pushed
84c2eced33fa: Pushed
2050af8ec892: Pushed
302e3ee49805: Pushed
1ccc411d2ebf: Pushed
02a08b0dd4f5: Pushed
f5c07c75b9d9: Pushed
latest: digest: sha256:98f90da3baee7c8be8bfd3685d97a208583c15e655f2811ef46d3ad8bf9839c size: 856
```



10.Login to Docker Hub using the following command to pull the image to Docker Hub:

```
D:\docker_assignment>docker pull kuppusamy71772118106/docker_assignment:latest
latest: Pulling from kuppusamy71772118106/docker_assignment
Digest: sha256:98f90da3baee7c8be8bfd3685d97a208583c15e655f2811ef46d3ad8bf9839c
Status: Image is up to date for kuppusamy71772118106/docker_assignment:latest
docker.io/kuppusamy71772118106/docker_assignment:latest

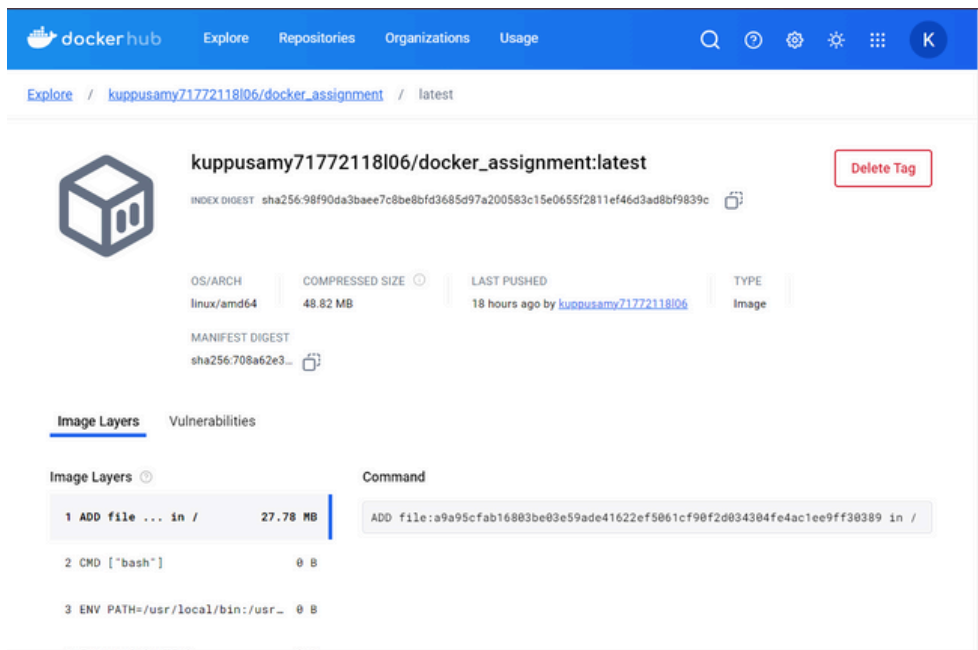
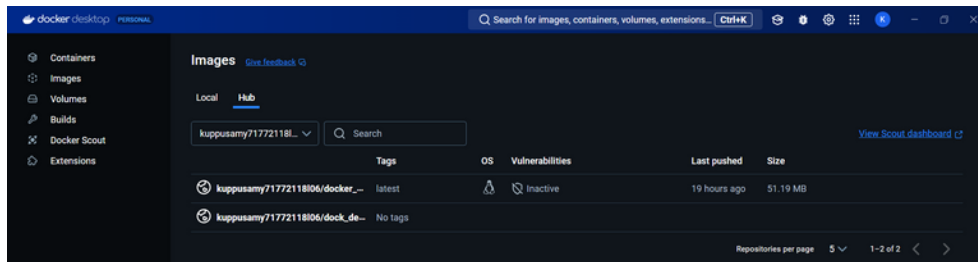
What's next:
View a summary of image vulnerabilities and recommendations + docker scout quickview kuppusamy71772118106/docker_assignment:latest
```

Docker image remove from container command: docker rm <image_name>

```
D:\docker_assignment>docker rm silly_brown
silly_brown
D:\docker_assignment>
```

11. Then I tagged the image for Docker Hub as seen in the following screenshot:

The below screenshot shows the docker images in Docker Hub:



- Push operation sends your locally built Docker image to the remote registry.
- Pull operation fetches images from a remote registry to your local system.
- Both operations allow efficient image management and sharing between local environments and Docker Hub.

GIT_GITHUB:https://github.com/Kuppu-samy/kuppusamy_2118l06

DOCKER_HUB:<https://hub.docker.com/repositories/kuppusamy71772118l06>